```python
# ------------------------------------------------------------------------------------
# File: main.py
# A menu Python program for students that uses strings, lists, tuples, and dictionaries.
# It also demonstrate the CRUD operations.
# ------------------------------------------------------------------------------------

student_db = {}
# Dictionary: This acts as the database for all student information.

# String: Example of these are name or ID.
def add_student(student_id, name, email, phone):
    """CREATE: Adds a new student to the database."""
    contact_info = (email, phone)
# Tuple: The contact details of students are in a form of tuple.

    grades = []
# List: The list of grades will be constantly updated.

    student_db[student_id] = {
        "name": name,
        "contact": contact_info,
        "grades": grades
    }

def view_student(student_id):
    """READ: Display the info of the student with the give ID."""
    if student_id in student_db:
        student = student_db[student_id]
        print(f"\n[ID: {student_id}] Name: {student['name']}")
        print(f"Contact: {student['contact']}")
        print(f"Grades: {student['grades']}")
        return student
    print("Error: Student ID not found.")
    return None

def update_grades(student_id, new_grade):
    """UPDATE: This will update a specific student's grade."""
    if student_id in student_db:
        student_db[student_id]["grades"].append(new_grade)
        return True
    return False

def display_all():
    """DISPLAY: This will display all the student data from the database."""
    print("\n--- Current Students ---")
    for sid, info in student_db.items():
        print(f"ID: {sid} | Name: {info['name']} | Grades: {info['grades']}")

def main_menu():
    """The Interactive Menu Loop."""
    while True:
        print("\n=== STUDENT RECORD SYSTEM ===")
        print("1. Add Student")
        print("2. View Student")
        print("3. Add Grade")
        print("4. Display All")
        print("5. Exit Application")

        choice = input("Select an option (1-5): ")

        if choice == '1':
            s_id = input("Enter ID: ")
            name = input("Enter Name: ")
            email = input("Enter Email: ")
            phone = input("Enter Phone: ")
            add_student(s_id, name, email, phone)
            print("Student added successfully!")

        elif choice == '2':
            s_id = input("Enter ID to search: ")
```

```python
            view_student(s_id)

        elif choice == '3':
            s_id = input("Enter Student ID: ")
            grade = input("Enter Grade: ")
            if update_grades(s_id, grade):
                print("Grade updated!")
            else:
                print("Student not found.")

        elif choice == '4':
            display_all()

        elif choice == '5':
            print("Exiting application. Goodbye!")
            break
        else:
            print("Invalid choice, try again.")

# They will be able to choose what they want in OneCompiler:
# either T for testing or M for the interactive menu.
if __name__ == "__main__":
    mode = input("Type 'T' for Tests or 'M' for Menu: ").upper()
    if mode == 'T':
# OneCompiler workaround
        import os
        os.system("python3 tests.py")
    else:
        main_menu()
```

The main file has a menu that is interactive and can loop. The user has either two options at the start: either to test or use the interactive menu of the main file. For the database, it uses a dictionary for the student information and tuples for the contact info as well as lists for the grades. When M is inputted, the menu pops out and tells the user to either press 1 to 5 wherein 1 adds a student based on the parameters, 2 views a student based on ID, 3 for updating the grades, 4 for displaying all information in the database, and 5 for exiting the menu.

It also has CRUD that creates students, reads their information, updates their grades and displays all their information.

```python
# ==========================================
# File: tests.py
# Description: Automated tests for Student Record System
# ==========================================

import main

def run_tests():
    print("--- Running Lab 2 Automated Tests ---")
    main.display_all()
    # Test 1: Create
    main.add_student("2022103327", "Nathaniel", "njcreynoso@mymail.mapua.edu.ph", "911")
    assert "2022103327" in main.student_db, "Test 1 Failed"
    main.display_all()
    print("[TEST 1] Student Creation: Passed")

    # Test 2: Update
    main.update_grades("2022103327", "90")
    student = main.student_db["2022103327"]
    assert "90" in student["grades"], "Test 2 Failed"
    main.display_all()
    print("[TEST 2] Grade Updating: Passed")

    # Test 3: Display
    db = main.student_db
    assert len(db) == 1, "Test 3 Failed"
    main.display_all()
    print("[TEST 3] Dictionary Retrieval: Passed")

    print("\nSUCCESS: All tests completed! OK")

if __name__ == '__main__':
    run_tests()
```

When they pressed T, it went to this file and used three test cases. The first test case checks if the main file can create and add a student. The next case is if it can truly update a grade from a student. Lastly, the third case is if they can display all information on the student record system.

STDIN

```
M
1
111
Nicole
Nicole@email.com
555-0911
```

Output: 10 ms | 9.5 MB

```
Type 'T' for Tests or 'M' for Menu:
=== STUDENT RECORD SYSTEM ===
1. Add Student
2. View Student
3. Add Grade
4. Display All
5. Exit Application
Select an option (1-5): Enter ID: Enter Name: Enter Email: Enter Phone: Student added successfully!

=== STUDENT RECORD SYSTEM ===
1. Add Student
2. View Student
3. Add Grade
4. Display All
5. Exit Application
```

This shows the output for the menu interactive loop.

STDIN

```
T
```

Output: 20 ms | 10.2 MB

```
Type 'T' for Tests or 'M' for Menu:
--- Running Lab 2 Automated Tests ---

--- Current Students ---

--- Current Students ---
ID: 2022103327 | Name: Nathaniel | Grades: []
[TEST 1] Student Creation: Passed

--- Current Students ---
ID: 2022103327 | Name: Nathaniel | Grades: ['90']
[TEST 2] Grade Updating: Passed

--- Current Students ---
ID: 2022103327 | Name: Nathaniel | Grades: ['90']
[TEST 3] Dictionary Retrieval: Passed

SUCCESS: All tests completed! OK
```

This shows the output for the test cases.